RHEL 8 Infrastructure Lab Documentation

Prepared by: Mahfujur Rahman
Domain: mahfujidb.com
Server Hostname: main.mahfujidb.com
Platform: RHEL 8
Purpose: Enterprise Infrastructure Lab Environment

1. Project Overview

This project was developed to simulate a small enterprise Linux infrastructure environment using Red Hat Enterprise Linux 8 (RHEL 8). The lab environment includes several core networking and system administration services commonly used in enterprise networks.

The infrastructure was configured and tested to provide practical experience in Linux server administration, network services, troubleshooting, security configuration, virtualization, and service integration.

Services Implemented

- Basic system configuration
- Static IP configuration using nmcli
- Samba file sharing service
- Local YUM repository configuration
- DNS server configuration using BIND
- DHCP server configuration
- Apache Web Server (httpd)
- Mail Server (Postfix + Dovecot + Gmail SMTP Relay)
- Root login security alert system
- Windows client integration
- SELinux management and policy configuration
- Firewall configuration using firewalld
- Time synchronization using NTP/Chrony
- KVM virtualization environment
- Virt-Manager graphical virtualization management
- Windows Server 2022 virtual machine deployment

Project Objectives

- To understand enterprise Linux infrastructure deployment
- To configure and manage essential network services
- To implement secure mail infrastructure with external SMTP relay
- To configure IMAP/POP3 mail access using Dovecot
- To integrate Linux servers with Windows clients
- To implement security monitoring via root login alerts
- To practice troubleshooting techniques
- To understand SELinux and firewall security management
- To gain hands-on experience with system administration tasks
- To simulate a real-world enterprise networking environment
- To understand Linux-based virtualization technologies
- To deploy and manage Windows virtual machines using KVM

2. Environment Information

Component	Value
Operating System	RHEL 8
Hostname	main.mahfujidb.com
Domain	mahfujidb.com
DNS Server IP	192.168.0.3
DHCP Server IP	192.168.0.3
Web Server IP	192.168.1.3
Mail Server	Postfix + Dovecot
SMTP Relay	smtp.gmail.com:587
Web Server	Apache (httpd)
DNS Software	BIND
DHCP Software	ISC DHCP Server
File Sharing	Samba
Virtualization Platform	KVM + QEMU
Virtualization Manager	Virt-Manager
Guest Operating System	Windows Server 2022
Client OS	Windows

Installed Software / Packages

Service	Package Name
Network Management	NetworkManager
DNS Server	bind, bind-utils
DHCP Server	dhcp-server
Web Server	httpd
Mail Server	postfix, dovecot
Mail Utilities	mailx
SMTP Authentication	cyrus-sasl, cyrus-sasl-plain
File Sharing	samba, samba-client
Time Synchronization	chrony
Firewall Management	firewalld
SELinux Tools	policycoreutils-python-utils
Virtualization Packages	@virt
Virtualization GUI	virt-manager
Hypervisor Services	qemu-kvm, libvirt
Text Editor	vim

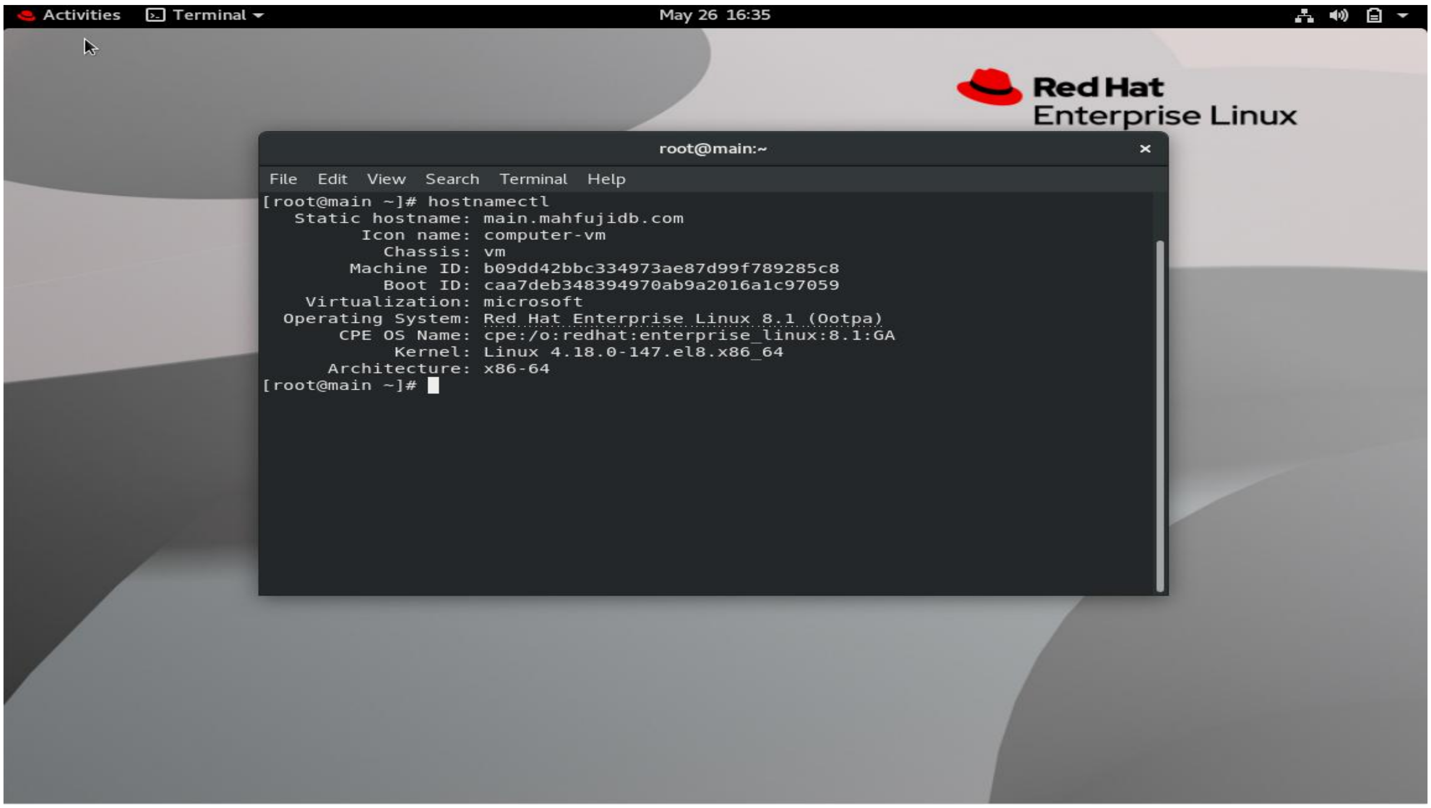
3. Initial System Configuration

Initial system configuration is the foundational step in deploying a secure and stable Linux server environment. In this project, the RHEL 8 server was configured with essential system settings including hostname assignment, network configuration, package updates, user management, and basic security settings. These configurations ensure proper communication, system stability, and readiness for enterprise service deployment.

3.1 Configure Hostname

Set hostname:
hostnamectl set-hostname main.mahfujidb.com

Verify:
hostnamectl
hostname –f

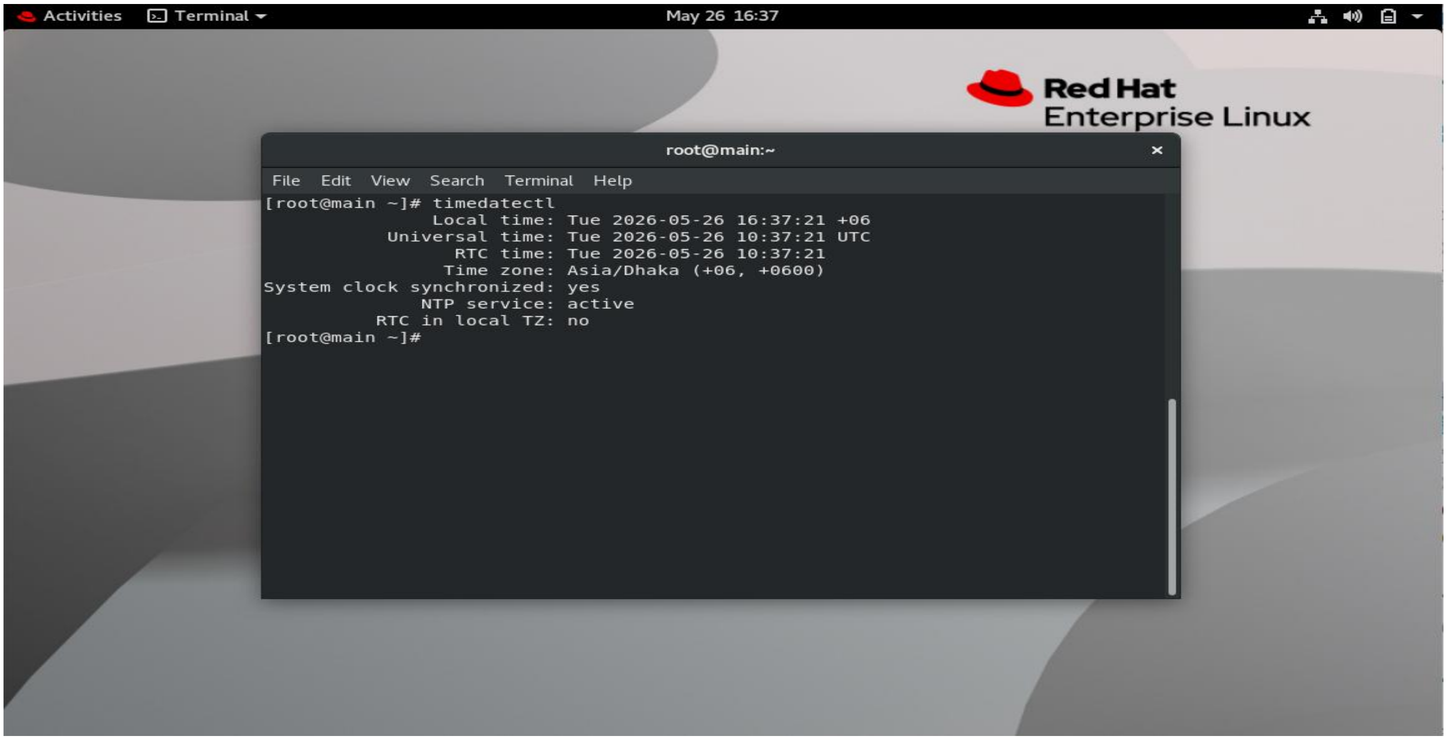


3.2 Configure Timezone

Set timezone:
timedatectl set-timezone Asia/Dhaka

Enable NTP synchronization:
timedatectl set-ntp true

Check timezone:
timedatectl

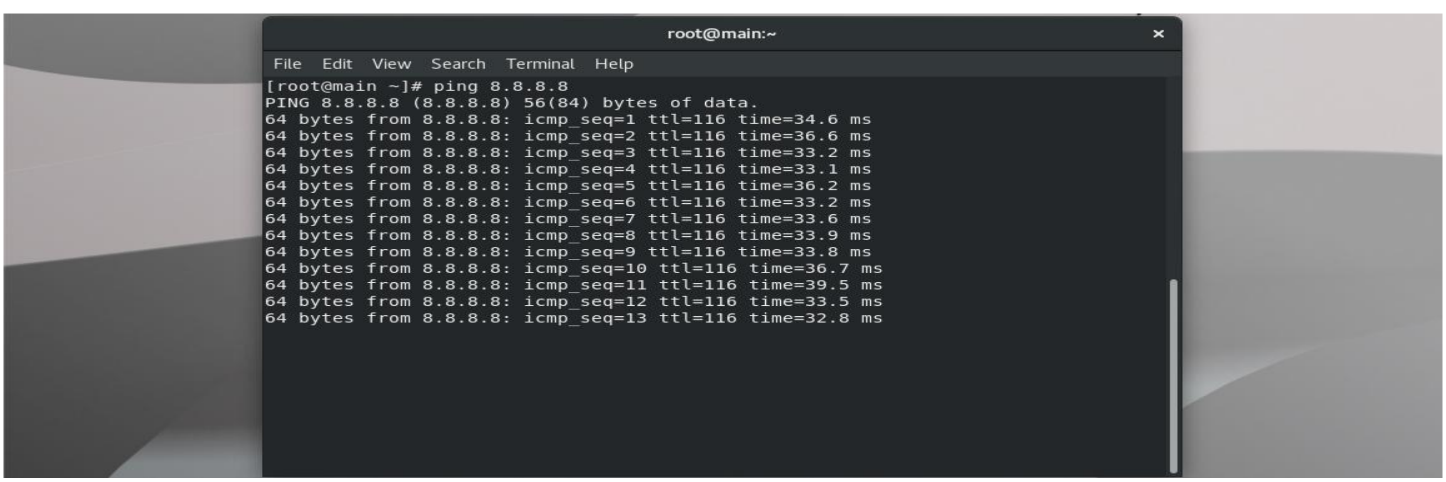


3.3 Static IP Assign and Verify Network Connectivity

sudo nmcli con mod eth0 ipv4.addresses 192.168.0.3/24
sudo nmcli con mod eth0 ipv4.gateway 192.168.0.1
sudo nmcli con mod eth0 ipv4.dns "8.8.8.8 1.1.1.1"
sudo nmcli con mod eth0 ipv4.method manual
sudo nmcli con down eth0
sudo nmcli con up eth0

Check IP address:
ip a

Check internet connectivity:
ping 8.8.8.8



4. Local YUM Repository Configuration

Local YUM repository configuration is the process of setting up an internal package repository to manage software installation and updates without relying on external internet sources. In this project, a local repository was configured on RHEL 8 to provide faster, stable, and offline package management. This setup improves system efficiency, ensures controlled software distribution, and supports enterprise-level Linux infrastructure environments.

4.1 Mount RHEL ISO

mount /dev/cdrom /mnt

Verify:

ls /mnt

4.2 Create Repository File

vim /etc/yum.repos.d/local.repo

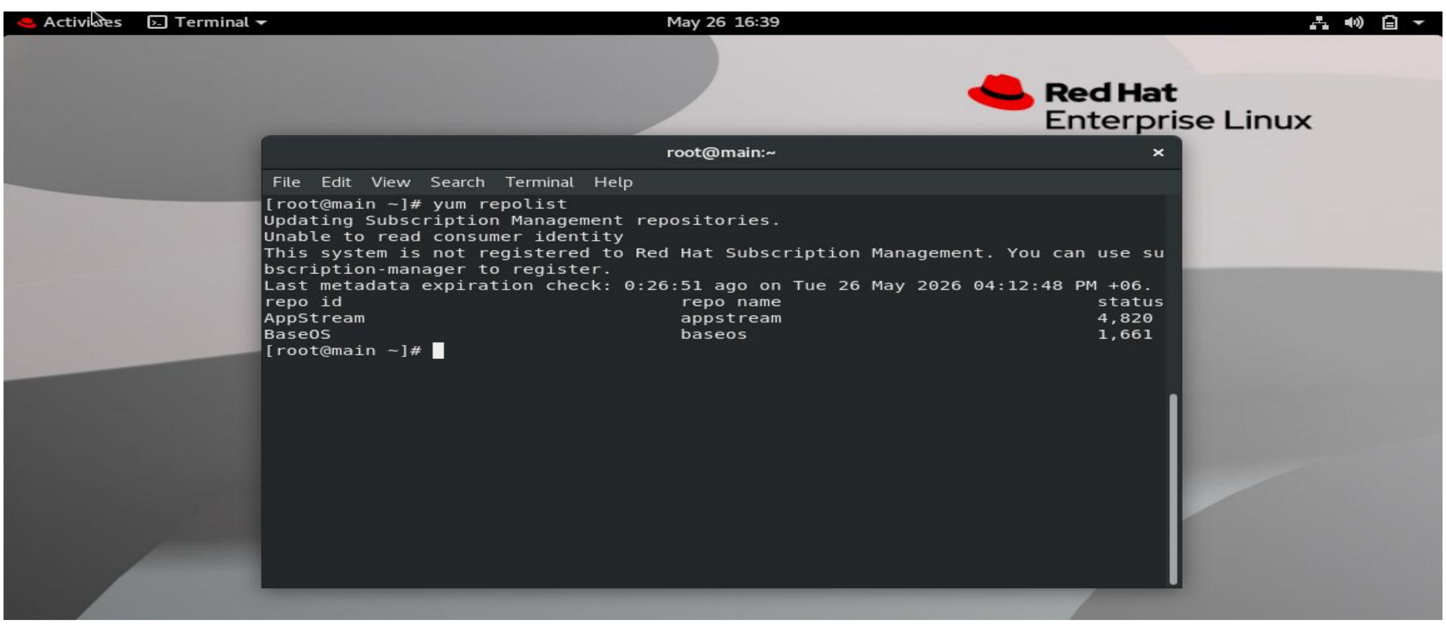
Add:

[BaseOS]
name=BaseOS
baseurl=file:///mnt/BaseOS
enabled=1
gpgcheck=0

[AppStream]
name=AppStream
baseurl=file:///mnt/AppStream
enabled=1
gpgcheck=0

4.3 Verify Repository

dnf repolist



5. Samba File Sharing Configuration

Samba file sharing configuration is the process of enabling secure file and resource sharing between Linux and Windows systems using the SMB protocol. In this project, Samba was configured on RHEL 8 to allow seamless data transfer and centralized file access within the network. This setup improves interoperability, simplifies file sharing between different operating systems, and supports efficient collaboration in enterprise environments.

5.1 Install Samba Packages

```
dnf install samba samba-client -y
```

5.2 Create Shared Directory

```
mkdir -p /shared
chmod 777 /shared
```

5.3 Configure Samba Share

Edit Samba configuration:

```
vim /etc/samba/smb.conf
```

Add the following section:

```
[shared]
path = /shared
browseable = yes
writable = yes
guest ok = yes
read only = no
```

5.4 Configure SELinux for Samba

By default, SELinux blocks Samba from accessing custom directories.

Assign SELinux context

```
semanage fcontext -a -t samba_share_t "/shared(/.*)?"
```

Apply SELinux context:

```
restorecon -Rv /shared
```

Enable Samba SELinux Boolean

```
setsebool -P samba_export_all_rw on
```

Verify:

```
getsebool samba_export_all_rw
```

Verify SELinux Label

ls -ldZ /shared

Expected type:

samba_share_t

5.5 Start Samba Services

systemctl enable smb
systemctl start smb

Verify:
systemctl status smb

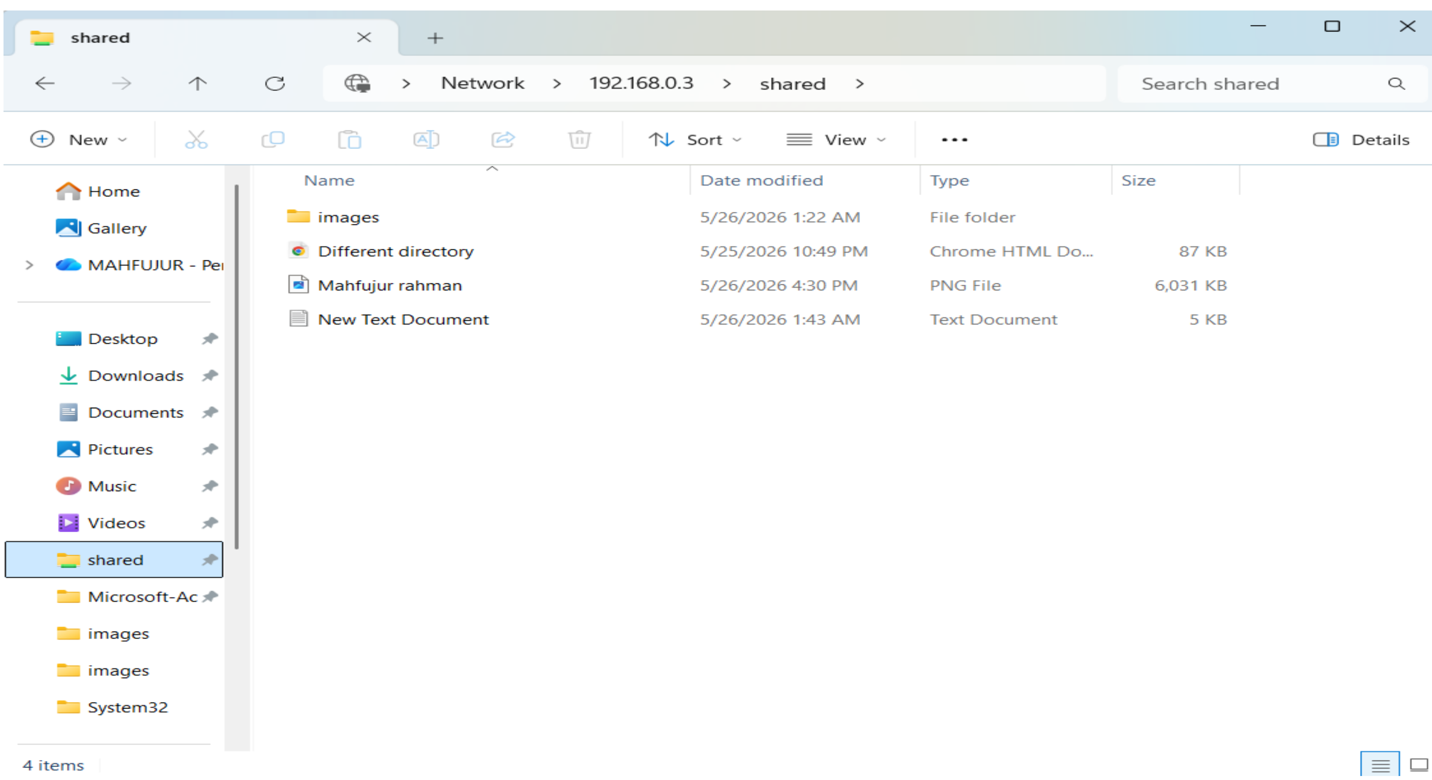
5.6 Configure Firewall for Samba

firewall-cmd --permanent --add-service=samba
firewall-cmd --reload

5.7 Access Samba Share from Windows

Open Run dialog:

[\\192.168.0.3\shared](\\192.168.0.3\shared)



6. DNS Server Configuration

DNS server configuration using BIND is the process of setting up a Domain Name System service to translate domain names into IP addresses within a network. In this project, BIND was configured on RHEL 8 to provide internal name resolution for the lab environment. This enables reliable host identification, improves network communication, and supports efficient access to services using domain names instead of IP addresses.

6.1 Install DNS Packages

```
dnf install bind bind-utils -y
```

6.2 Configure Main DNS File

Edit:

```
vim /etc/named.conf
```

Modify options:

```
options {
   listen-on port 53 { any; };
   listen-on-v6 port 53 { any; };
   directory "/var/named";
   allow-query { any; };
   recursion yes;
};
```

6.3 Add DNS Zones

Add below in `/etc/named.conf`:

```
zone "mahfujidb.com" IN {
   type master;
   file "forward.zone";
};

zone "0.168.192.in-addr.arpa" IN {
   type master;
   file "reverse.zone";
};
```

6.4 Configure Forward Zone

Create zone file:

```
vim /var/named/forward.zone
```

Add:

```
$TTL 86400
@ IN SOA ns1.mahfujidb.com. root.mahfujidb.com. (
2026052601
```

3600
1800
604800
86400)

@   IN NS ns1.mahfujidb.com.

@   IN A 192.168.0.3
ns1   IN A 192.168.0.3
main   IN A 192.168.0.3
www   IN A 192.168.1.3
mail   IN A 192.168.1.230

@   IN MX 10 mail.mahfujidb.com.

6.5 Configure Reverse Zone

vim /var/named/reverse.zone

Add:

$TTL 86400
@ IN SOA ns1.mahfujidb.com. root.mahfujidb.com. (
2026052601
3600
1800
604800
86400)

@ IN NS ns1.mahfujidb.com.

3 IN PTR main.mahfujidb.com.

6.6 Configure Permissions

chown root:named /var/named/forward.zone
chown root:named /var/named/reverse.zone

chmod 640 /var/named/forward.zone
chmod 640 /var/named/reverse.zone

6.7 Configure SELinux for DNS

restorecon -Rv /var/named
setsebool -P named_write_master_zones on

6.8 Verify DNS Configuration

named-checkconf

Check forward zone:
named-checkzone mahfujidb.com /var/named/forward.zone

Check reverse zone:
named-checkzone 0.168.192.in-addr.arpa /var/named/reverse.zone

6.9 Start DNS Service

systemctl enable named
systemctl restart named

Verify:
systemctl status named

6.10 Configure Firewall

firewall-cmd --permanent --add-service=dns
firewall-cmd --reload

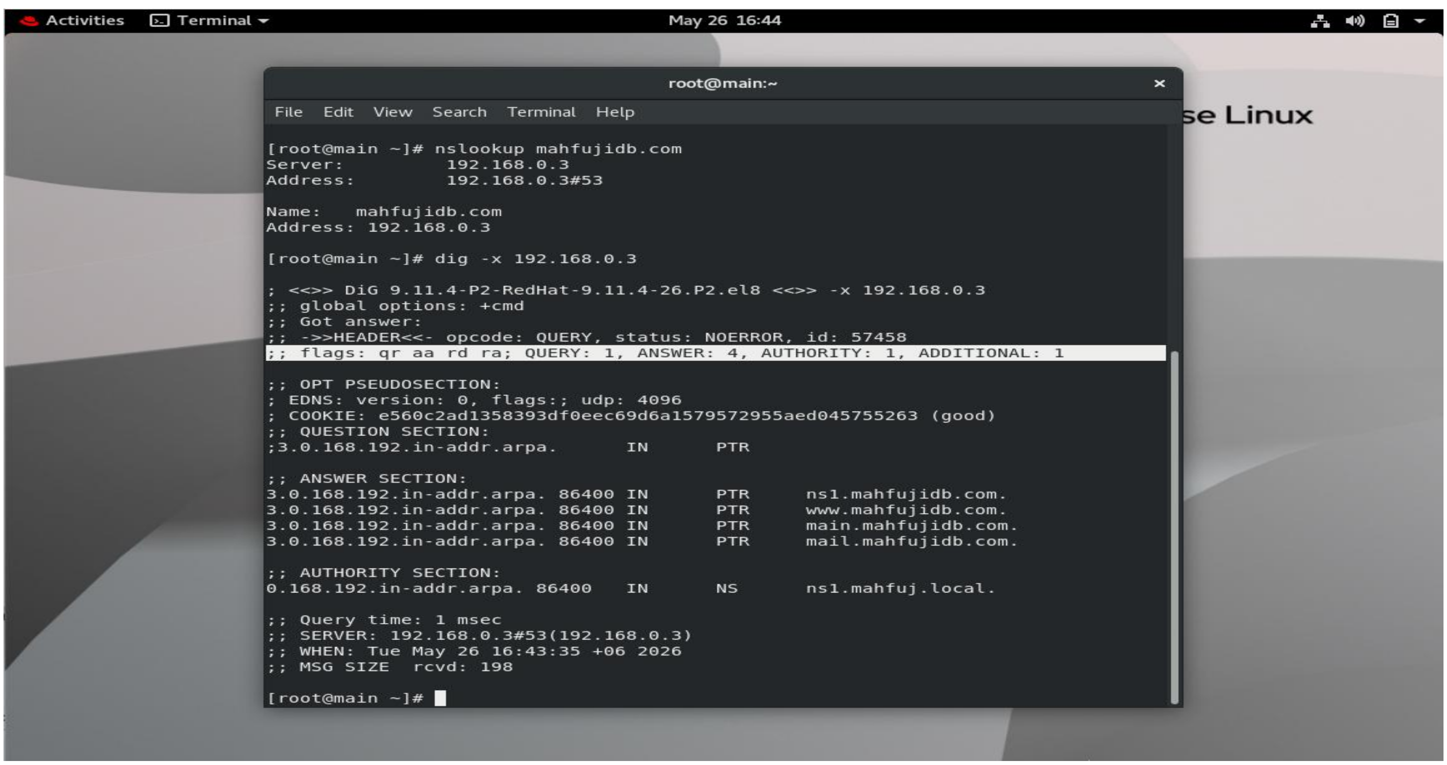
6.11 Test DNS

Forward lookup:

nslookup mahfujidb.com
nslookup www.mahfujidb.com

Reverse lookup:

dig -x 192.168.0.3



7. DHCP Server Configuration

DHCP server configuration is the process of setting up a Dynamic Host Configuration Protocol service to automatically assign IP addresses and network configuration details to client devices. In this project, a DHCP server was configured on RHEL 8 to provide dynamic IP allocation within the lab environment. This reduces manual configuration effort, prevents IP conflicts, and ensures efficient and centralized network management.

7.1 Install DHCP Server

```
dnf install dhcp-server -y
```

7.2 Configure DHCP

Edit:

```
vim /etc/dhcp/dhcpd.conf
```

Add:

```
ddns-update-style interim;
authoritative;

default-lease-time 600;
max-lease-time 7200;

subnet 192.168.0.0 netmask 255.255.255.0 {

   range 192.168.0.50 192.168.0.150;

   option routers 192.168.0.1;

   option subnet-mask 255.255.255.0;

   option domain-name "mahfujidb.com";

   option domain-name-servers 192.168.0.3;
}
```

7.3 Verify DHCP Configuration

```
dhcpd -t
```

7.4 Start DHCP Service

```
systemctl enable dhcpd
systemctl restart dhcpd
```

Verify:

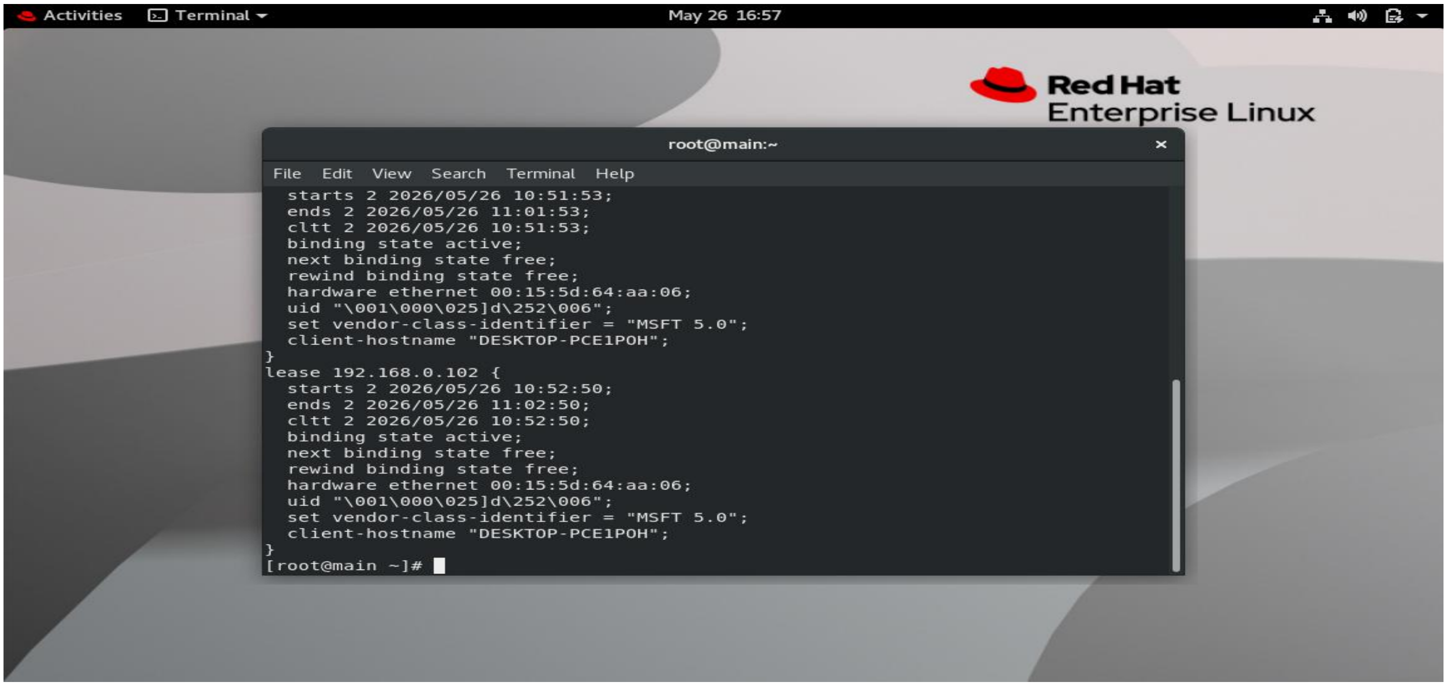
```
systemctl status dhcpd
```

7.5 Configure Firewall

firewall-cmd --permanent --add-service=dhcp
firewall-cmd --reload

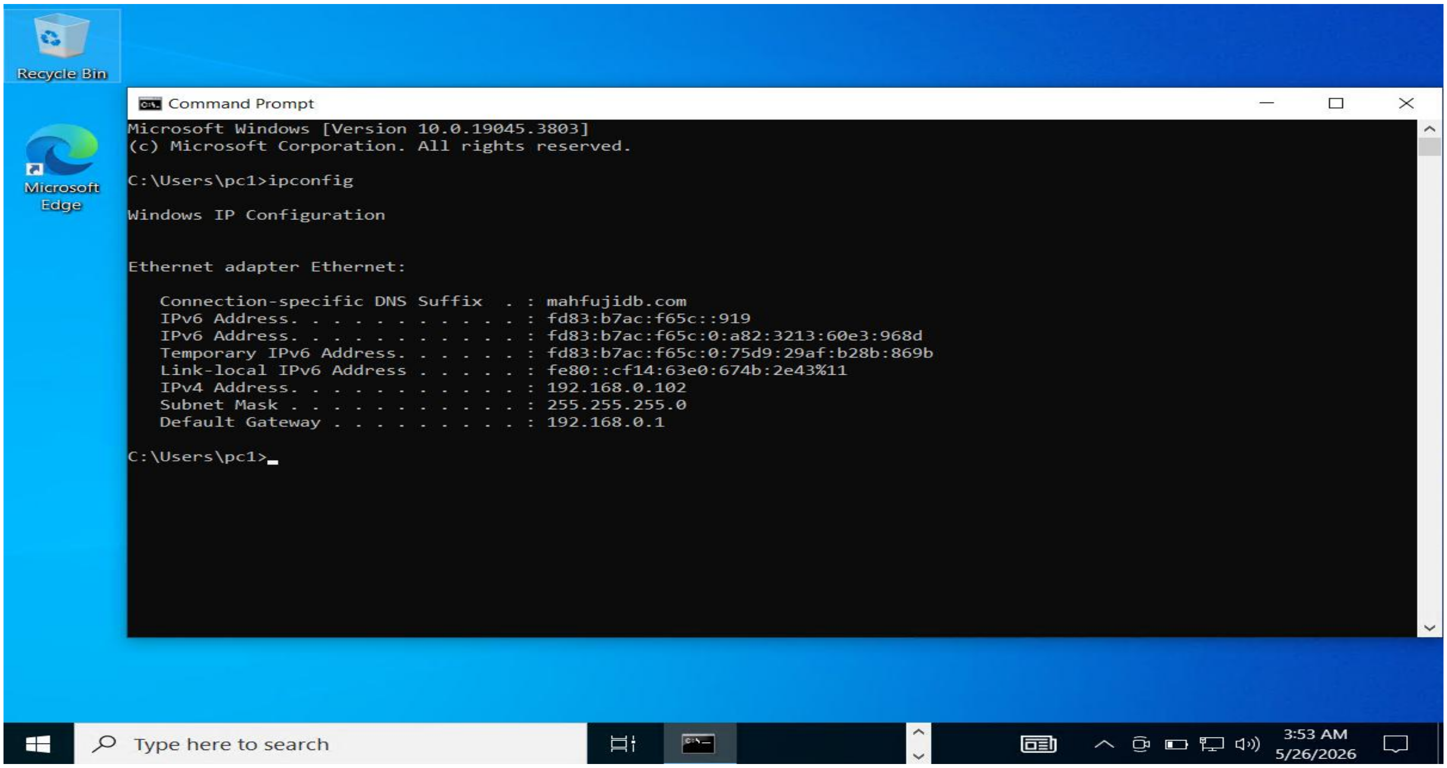
7.6 Verify DHCP Leases

cat /var/lib/dhcpd/dhcpd.leases



7.6 Verify DHCP Client

Ipconfig



8. Apache Web Server Configuration

Apache web server configuration is the process of installing and setting up the HTTP server to host and serve web content over a network. In this project, Apache (httpd) was configured on RHEL 8 to provide a functional web hosting environment within the lab infrastructure. This enables hosting of static and dynamic web pages, supports testing of web services, and simulates real-world enterprise web server deployment.

8.1 Install Apache

```
dnf install httpd -y
```

8.2 Start Apache

```
systemctl enable httpd
systemctl start httpd
```

8.3 Configure Firewall

```
firewall-cmd --permanent --add-service=http
firewall-cmd --reload
```

8.4 Create Website Directory

```
mkdir -p /var/www/mahfuj-site
```

Copy website files:

```
cp shared/index.html /var/www/mahfuj-site/index.html
cp shared/images/* /var/www/mahfuj-site/images
```

8.5 Configure Apache Virtual Host

Create configuration:

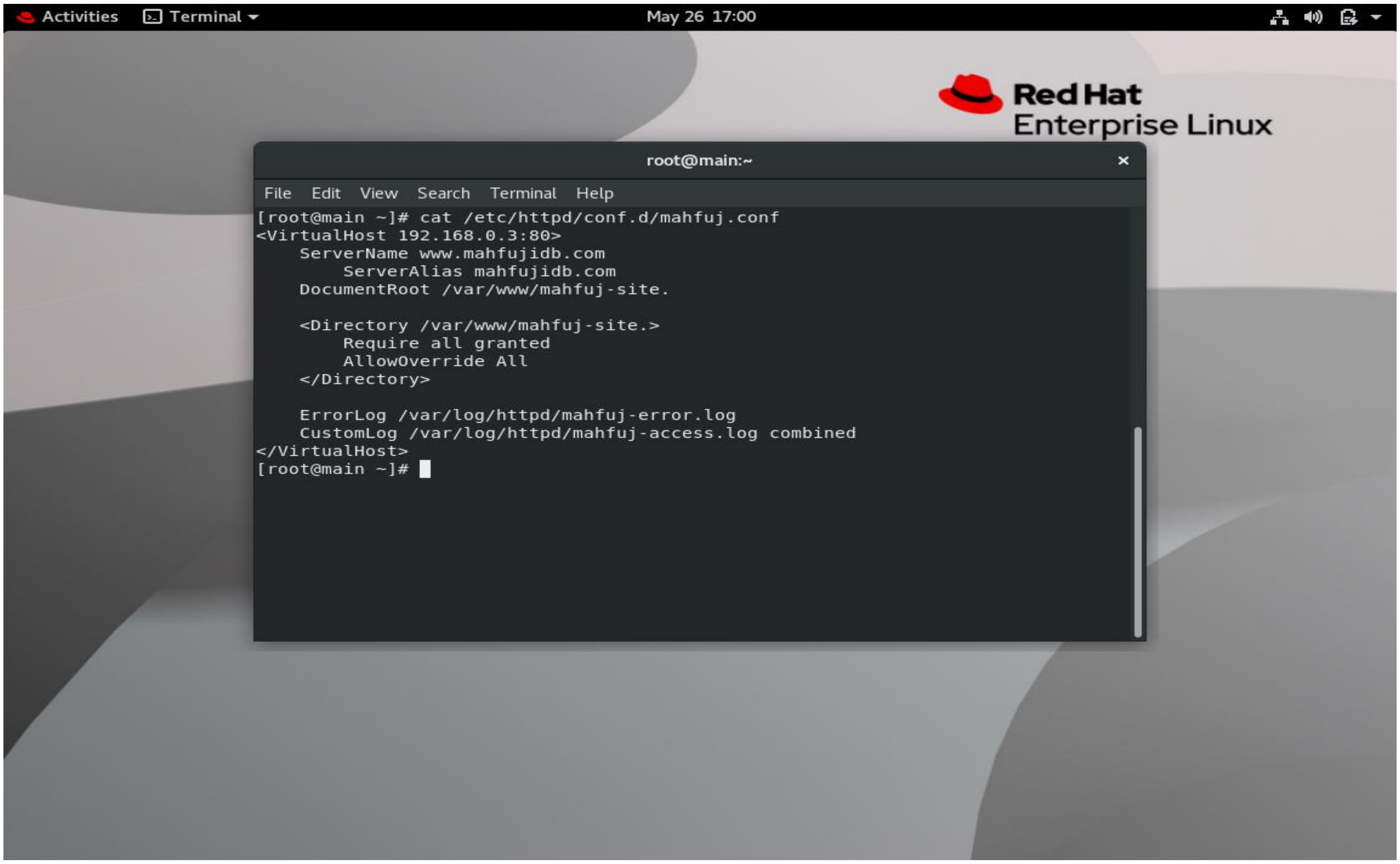
```
vim /etc/httpd/conf.d/mahfuj.conf
```

Add:

```
<VirtualHost 192.168.0.3:80>

  ServerName www.mahfujidb.com
  ServerAlias mahfujidb.com

  DocumentRoot /var/www/mahfuj-site

  <Directory /var/www/mahfuj-site>
    Require all granted
    AllowOverride All
  </Directory>
```

ErrorLog /var/log/httpd/mahfuj-error.log
CustomLog /var/log/httpd/mahfuj-access.log combined

</VirtualHost>



8.6 Configure Permissions

chown -R apache:apache /var/www/mahfuj-site
chmod -R 755 /var/www/mahfuj-site

8.7 Configure SELinux for Apache

restorecon -Rv /var/www/
setsebool -P httpd_read_user_content on

8.8 Restart Apache

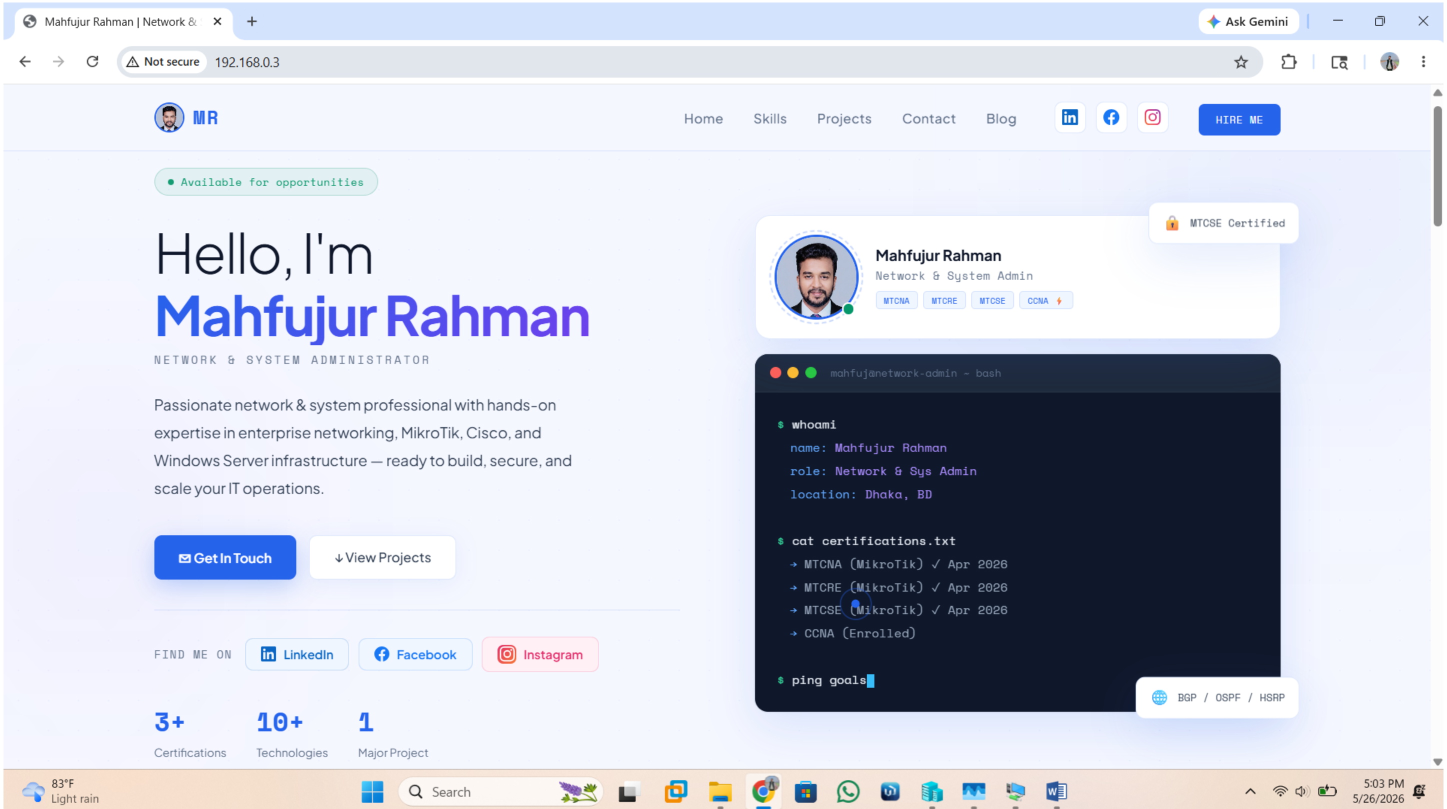
systemctl restart httpd

8.9 Verify Apache

curl http://localhost
apachectl -S

Browser test:

http://192.168.0.3



.html file link

https://drive.google.com/drive/folders/1HBe52g7USZak-yr8LSkjSEWHVTnD0SEj?usp=sharing

9. Virtualization Environment Configuration

Virtualization environment configuration is the process of setting up a hypervisor-based system to create and manage multiple virtual machines on a single physical host. In this project, KVM, QEMU, and libvirt were configured on RHEL 8, along with Virt-Manager for graphical management. This setup enables efficient resource utilization, supports multiple operating systems, and provides an enterprise-ready platform for virtual machine deployment and testing.

9.1 Install Virtualization Packages

KVM and virtualization tools were installed using:
dnf install -y @virt

Virt-Manager package installation:
dnf install -y virt-manager

9.2 Enable libvirt Service

The virtualization service was enabled and started:
systemctl enable --now libvirtd

Verify service status:
systemctl status libvirtd

9.3 Launch Virt-Manager

Virt-Manager graphical console was launched using:
virt-manager

The host system connected successfully to:
QEMU/KVM

9.4 ISO Image Preparation

The Windows Server 2022 ISO file was stored using smb-share in:
/shared

Example:
/shared/Windows_Server_2022.iso

9.5 Virtual Machine Creation

A new virtual machine was created using Virt-Manager.

VM Configuration

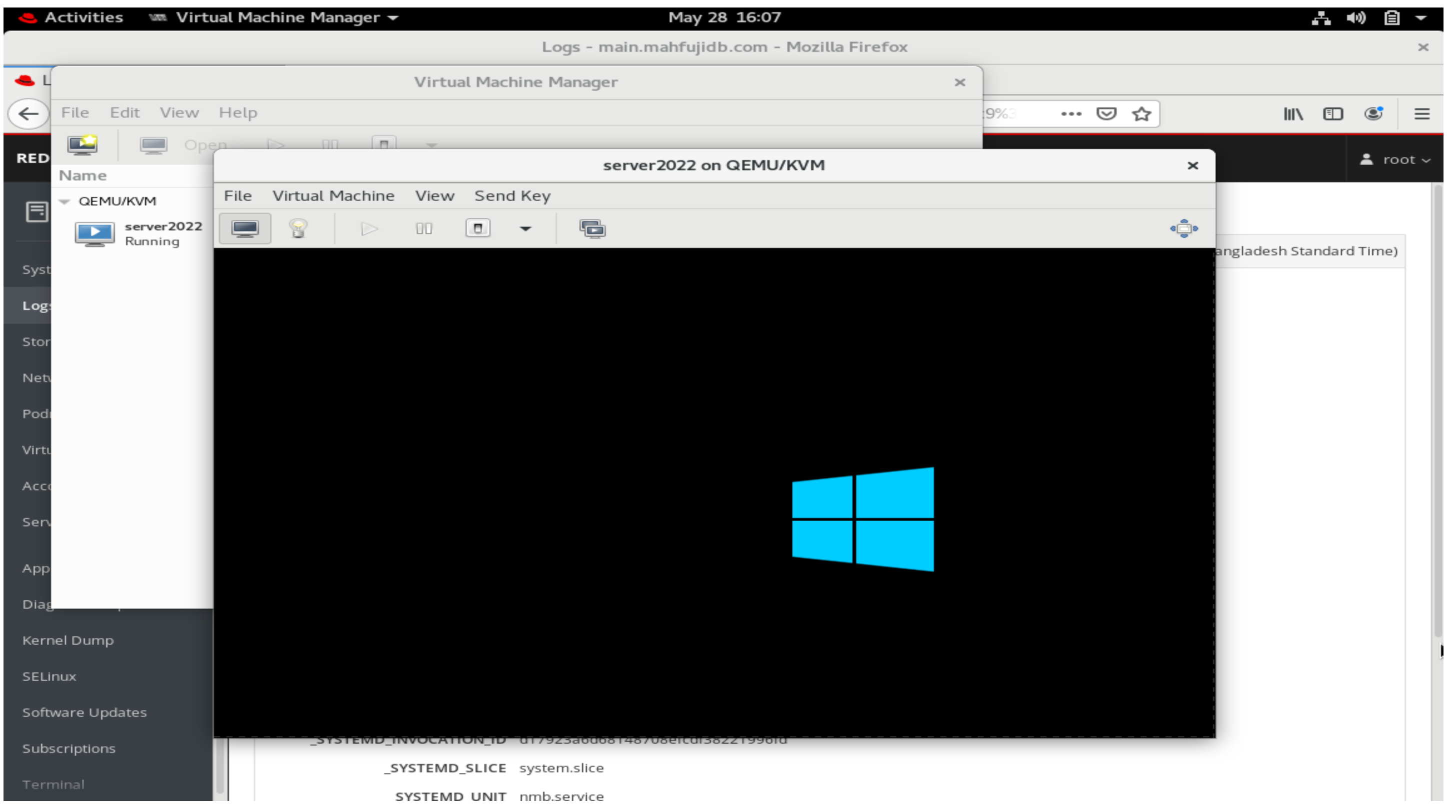
Resource	Configuration
VM Name	server2022

Resource	Configuration
RAM	2 GB
vCPUs	2
Disk Size	40 GB
Disk Format	QCOW2
Network	Default NAT
Installation Media	Windows Server 2022 ISO

9.6 Virtual Disk Creation

The virtual disk was automatically created at:
/var/lib/libvirt/images/server2022.qcow2

Disk verification command:
qemu-img info /var/lib/libvirt/images/server2022.qcow2



9.7 Windows Server 2022 Installation Process

The installation process included:

1. Boot VM from ISO image
2. Select language and regional settings
3. Start Windows installation
4. Select Windows Server 2022 edition
5. Configure virtual disk

6. Complete operating system installation
7. Configure Administrator password
8. Perform initial server setup

Installation completed successfully.

9.8 Virtual Machine Management

List Virtual Machines
virsh list --all

Start Virtual Machine
virsh start server2022

Shutdown Virtual Machine
virsh shutdown server2022

10. Mail Infrastructure Configuration (Postfix, Dovecot & Security Alerts)

Mail infrastructure configuration involves setting up a complete email system that supports sending, receiving, and securely managing email services within a network. In this project, Postfix was configured as the SMTP server and Dovecot was used for IMAP/POP3 services on RHEL 8, along with Gmail SMTP relay integration for external email delivery. Additionally, a security alert system was implemented to monitor and notify root login activities. This setup provides a secure and functional enterprise mail environment suitable for communication and system monitoring.

10.1 Install Required Packages

Install Postfix, Dovecot, and mail utilities:
dnf install postfix dovecot mailx -y

10.2 Enable and Start Services

Enable services at boot:
systemctl enable postfix
systemctl enable dovecot

Start services:
systemctl start postfix
systemctl start dovecot

Check status:
systemctl status postfix
systemctl status dovecot

10.3 Configure Gmail SMTP Relay (Postfix)

Edit main configuration:
vim /etc/postfix/main.cf

Add/modify:
myhostname = main.mahfujidb.com
mydomain = mahfujidb.com
myorigin = $mydomain
inet_interfaces = all
inet_protocols = ipv4
mydestination = $myhostname, localhost.$mydomain, localhost, $mydomain
mynetworks = 127.0.0.0/8, 192.168.0.0/24
home_mailbox = Maildir/
relayhost = [smtp.gmail.com]:587

10.4 Enable SMTP Authentication (Gmail)

Create auth file:
vim /etc/postfix/sasl_passwd

Add:
[smtp.gmail.com]:587 mahfujtanvirdm@gmail.com:app_password

Secure and compile:
postmap /etc/postfix/sasl_passwd
chmod 600 /etc/postfix/sasl_passwd
chmod 600 /etc/postfix/sasl_passwd.db

Add authentication settings to Postfix:
vim /etc/postfix/main.cf

Add:
smtp_sasl_auth_enable = yes
smtp_sasl_password_maps = hash:/etc/postfix/sasl_passwd
smtp_sasl_security_options = noanonymous
smtp_use_tls = yes
smtp_tls_security_level = encrypt
smtp_tls_CAfile = /etc/pki/tls/certs/ca-bundle.crt
setgid_group = postdrop

Restart Postfix:
systemctl restart postfix

10.5 Configure Dovecot (IMAP/POP3)

Enable protocols:
vim /etc/dovecot/dovecot.conf

Add:
protocols = imap pop3 lmtp

Set Mail location:
vim /etc/dovecot/conf.d/10-mail.conf

Add:
mail_location = maildir:~/Maildir

Enable login authentication:
vim /etc/dovecot/conf.d/10-auth.conf

Set:
disable_plaintext_auth = no
auth_mechanisms = plain login

10.6 Restart Dovecot:

systemctl restart dovecot

10.7 Create Maildir Structure (Users)

For mail delivery:
mkdir -p ~/Maildir/{cur,new,tmp}

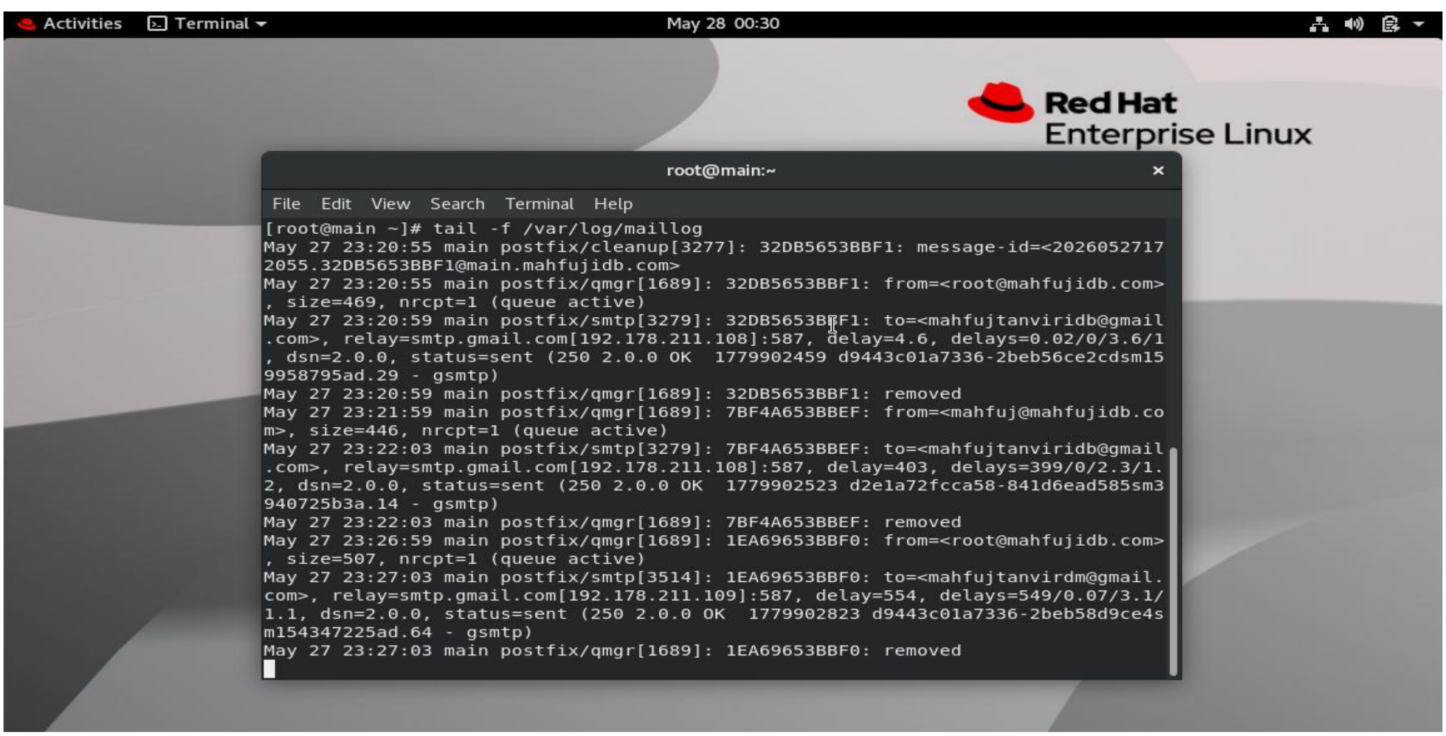
Set permissions:
chmod -R 700 ~/Maildir

10.8 Test Mail Sending Locally

Send test email:
echo "mail server test" | mail -s "test mail" mahfuj

10.9 Verify Mail Logs

Check delivery status:
tail -f /var/log/maillog



10.10 Root Login Alert System (Automation)

Step 1: Edit root bash profile

```
vim /root/.bashrc
```

Step 2: Add Alert Script

Add at the bottom:

```
echo "ALERT - Root Login on $(hostname) at $(date) from $(who am i | awk '{print $5}')" \
| mail -s "ROOT LOGIN ALERT" mahfujtanvirdm@gmail.com
```

Step 3: Activate Immediately

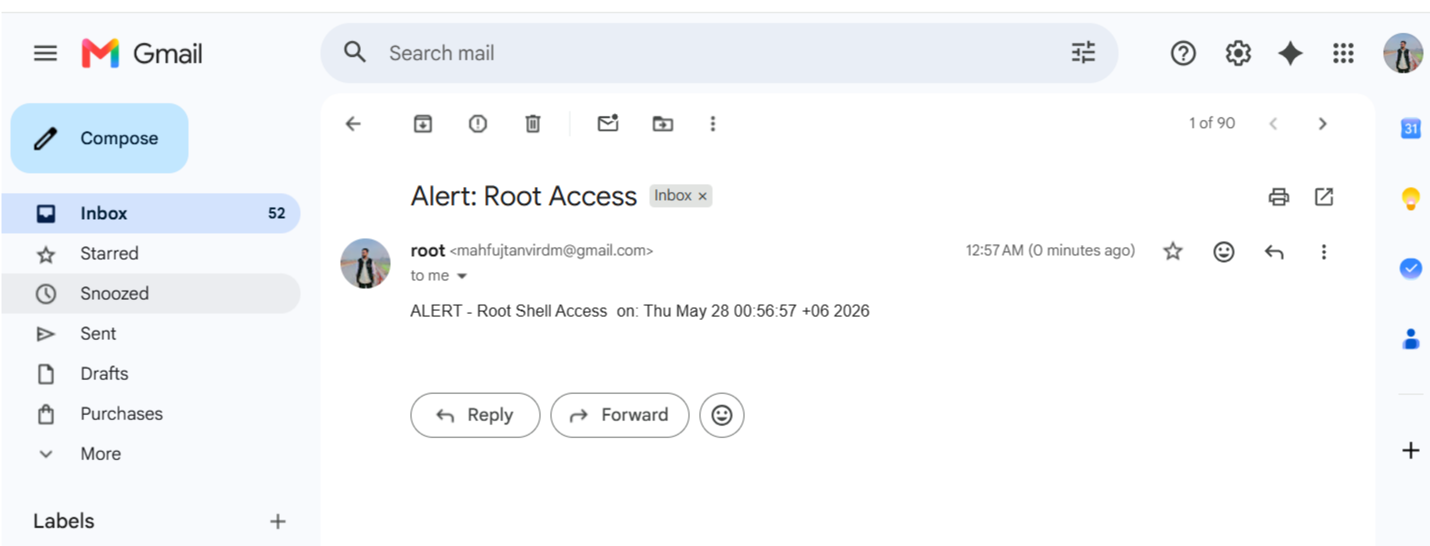
```
source /root/.bashrc
```

Step 4: Working Flow

When root login happens:

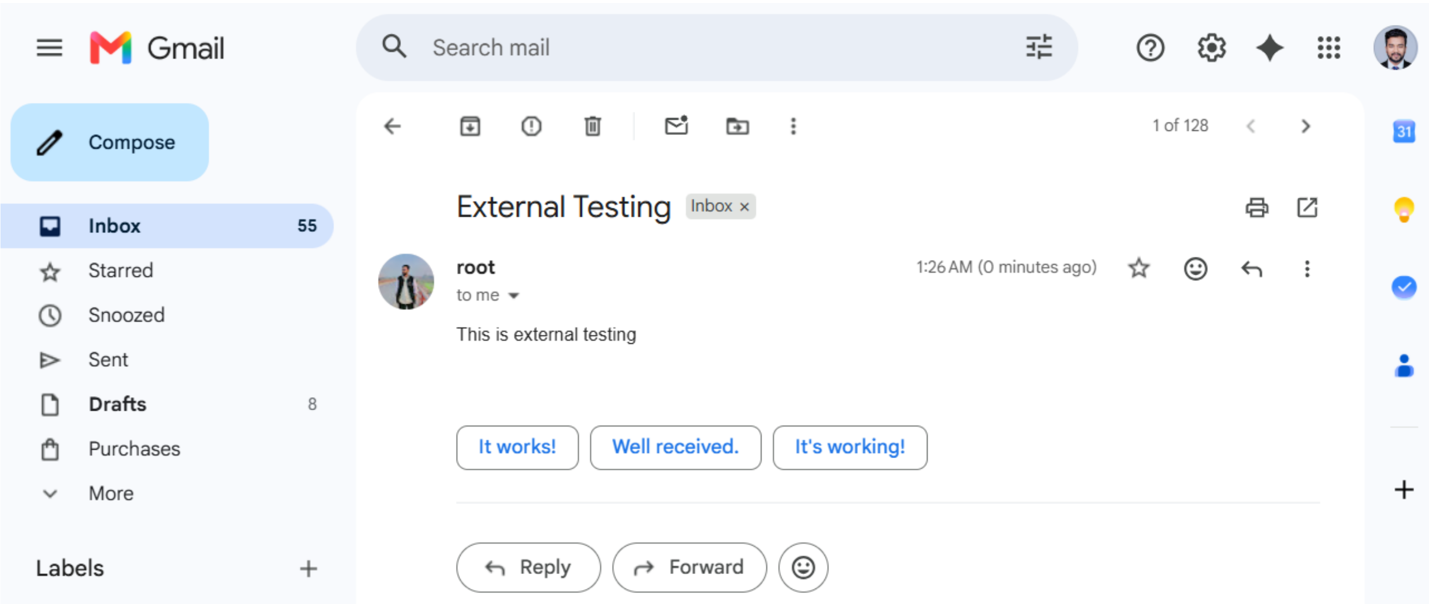
1. .bashrc executes automatically
2. System captures:
   - Hostname
   - Time/date
   - Source IP
3. Mail command triggers alert
4. Postfix sends via Gmail SMTP
5. Email delivered to Gmail inbox

Verify
Source .bashrc



10.11 External Testing

echo "This is external testing" | mail -s "External Testing" mahfujtanviridb@gmail.com



11. Conclusion

This RHEL 8 infrastructure lab successfully demonstrates the deployment, configuration, and integration of essential enterprise network, virtualization, and system administration services within a Linux-based environment.

The project includes the implementation of:

- Samba file sharing
- Local package management (YUM Repository)
- DNS server configuration using BIND
- DHCP server configuration
- Apache web hosting services
- Mail infrastructure using Postfix and Dovecot
- Gmail SMTP relay integration
- Root login security alert system
- SELinux policy and security management
- Firewall administration using firewalld
- KVM virtualization environment
- Virt-Manager graphical virtualization management
- Windows Server 2022 virtual machine deployment
- Windows and Linux system integration

The lab environment provides practical hands-on experience in Linux system administration, enterprise infrastructure deployment, service integration, troubleshooting, virtualization management, and security configuration.

This project also strengthens understanding of real-world enterprise networking and virtualization concepts while improving the ability to deploy, manage, secure, and maintain Linux-based and virtualized server infrastructures.